

Auditix AI — instalar no PC de casa

Do zero até a câmera detectando queda e o alerta chegando no e-mail. Grupo 11 · SVATech

O que você NÃO precisa refazer. Tudo o que está na AWS já existe e vale para qualquer máquina: a função `aviso-next`, a URL dela, o e-mail verificado no SES, as permissões e o banco PostgreSQL. Nenhum computador “tem” a Lambda — todos apenas chamam a mesma URL.

- Na sua máquina nova só falta: os programas, o projeto e o **arquivo** `.env`.

1 Instalar os programas

Dois, e só se ainda não existirem na máquina:

PROGRAMA	PARA QUE SERVE / COMO INSTALAR
Git	Baixar e atualizar o projeto. <code>git-scm.com/download/win</code> — instalador comum, tudo “Next”.
uv	Roda o Python sem instalar Python. No PowerShell: <code>irm https://astral.sh/uv/install.ps1 iex</code> Feche e abra o PowerShell depois , senão o comando <code>uv</code> não é encontrado.

Confira que os dois responderam:

```
git --version
uv --version
```

2 Baixar o projeto

Escolha onde a pasta vai ficar (ex.: a Área de Trabalho) e:

```
cd $HOME\Desktop
git clone https://github.com/EnzoRenato16/Next.git
cd Next
git checkout claude/entender-projeto-0tgk9u
```

Se a pasta já existir nessa máquina, não clone de novo — entre nela e atualize:

```
cd $HOME\Desktop\Next
git pull
```

3 Criar o arquivo `.env`

Esta é a única parte que o GitHub não traz, **de propósito**: o `.env` guarda senha e segredo, e senha em repositório é senha vazada. Por isso cada máquina cria o seu.

Na pasta do projeto, no PowerShell. Repare que a primeira linha é `Set-Content` (cria o arquivo) e as outras são `Add-Content` (acrescentam). Se trocar, apaga tudo.

```
Set-Content .env "DATABASE_URL=postgresql://postgres:SENHA_DO_BANCO@db-fiap.cdq26s2ayupv.us-east-2.rds.amazonaws.com:5432/postgres?sslmode=require"
Add-Content .env "WEBHOOK_URL=https://73sq6kkyjuyh3aowq3vo2w4t2m0rpxbd.lambda-url.us-east-1.on.aws/"
Add-Content .env "WEBHOOK_SEGREDO=0_SEGREDO_DA_LAMBDA"
Add-Content .env "ALERTA_TIPOS=graves"
Add-Content .env "ALERTA_ESPERA=60"
Get-Content .env
```

O `Get-Content` tem que mostrar **cinco linhas, sem nenhuma repetida**, e cada uma começando com o nome da variável seguido de `=`, sem espaços em volta.

VARIÁVEL	O QUE É
<code>DATABASE_URL</code>	Banco PostgreSQL. Sem ela o sistema funciona igual, mas grava num arquivo local (SQLite) só daquela máquina.
<code>WEBHOOK_URL</code>	A URL da função <code>avisos-next</code> . Já está preenchida acima.
<code>WEBHOOK_SEGREDO</code>	Tem que ser idêntico à variável <code>SEGREDO</code> da Lambda. Diferente = erro 403 e nenhum e-mail.
<code>ALERTA_TIPOS</code>	<code>graves</code> em uso normal. <code>todos</code> manda e-mail de tudo — serve para testar, é insuportável no dia a dia.
<code>ALERTA_ESPERA</code>	Silêncio mínimo, em segundos, entre dois e-mails do mesmo tipo.

Os dois valores secretos não estão escritos neste PDF.

A senha do banco e o segredo da Lambda ficam só com você — um PDF passa de mão em mão, vai para o grupo, para o e-mail do professor. Pegue a senha do banco com quem criou a instância, e o segredo em **Lambda → avisos-next → Configuração → Variáveis de ambiente**, na chave `SEGREDO`.

Não use o Bloco de Notas para criar o `.env`.

Ele salva como `.env.txt` sem avisar, e o servidor nunca lê esse arquivo. Se desconfiar, rode `Get-ChildItem -Force .env*`: se aparecer `.env.txt`, ou tamanho `0`, é isso.

4 Subir o servidor

```
uv run servidor.py
```

Na primeira vez demora um pouco: o `uv` baixa o Python e as bibliotecas. Depois abre em segundos. As linhas da partida dizem se está tudo certo:

```
[auditix] PostgreSQL conectado: db-fiap...us-east-2
[auditix] sala -> http://127.0.0.1:8000/
[auditix] painel -> http://127.0.0.1:8000/painel
[auditix] alerta por e-mail LIGADO (graves, espera 60s)
```

SE APARECER...	É ISSO
Caindo para SQLite local	O DATABASE_URL está errado ou a senha não confere. O sistema continua funcionando, só não grava na nuvem.
alerta por e-mail DESLIGADO	Falta o WEBHOOK_URL no .env — ou o arquivo virou .env.txt.
SEM SEGREDO na mesma linha	A linha do WEBHOOK_SEGREDO está sem o nome da variável na frente.

Deixe essa janela aberta. Fechar ela derruba o sistema. Ctrl+C encerra.

5 Abrir a Sala

No navegador: `http://127.0.0.1:8000/`

Com a **webcam** do notebook funciona na hora — é o suficiente para testar tudo. Se for usar a câmera IP, veja o passo 6 antes.

Espere as 6 etapas de carregamento terminarem. Os modelos de rosto e de corpo vêm da própria pasta do projeto, então **não precisa de internet** para a Sala abrir.

6 Câmera IP (opcional)

Navegador nenhum abre RTSP. Quem traduz é o **go2rtc**, um executável só, sem instalador: `github.com/AlexxIT/go2rtc/releases` → `go2rtc_win64.zip`.

Deixe o `.exe` numa pasta com um arquivo `go2rtc.yaml` ao lado:

```
api:
  listen: ":1984"
  origin: "*"          # sem isto o navegador recusa a conexão

streams:
  sala: rtsp://USUARIO:SENHA@192.168.0.244:554/stream1
```

Dois cliques no `go2rtc.exe`, deixe a janela aberta, confira em `http://127.0.0.1:1984`. Depois, na Sala, clique em **“fonte de vídeo: webcam”** e escreva `127.0.0.1:1984/sala`.

Onde pôr a câmera: canto da sala, 2,5 a 3 m, inclinada 30° a 45° para baixo. A regra prática: se dá para ver o **quadril** das pessoas, está bom. Câmera no teto olhando para baixo não serve — em pé e caído ocupam quase o mesmo lugar no quadro.

7 Marcar a linha do chão

Com a câmera fixa, o chão fica sempre na mesma altura da imagem. O botão **Linha do chão** deixa marcar essa altura clicando na própria imagem — e daí em diante **cabeça abaixo da linha é queda**, direto, sem depender do ângulo do tronco.

Marque na altura em que a cabeça de alguém **deitado no chão** estaria: um pouco acima do chão visível no fundo da imagem.

A linha vale só para esta câmera, nesta posição.

Mexeu nela, marque de novo. E a marcação fica salva no navegador, não no projeto: em outra máquina, ou depois de limpar o navegador, é preciso refazer.

8 Testar o alerta

Sem precisar cair no chão. Em **outra** janela do PowerShell, na pasta do projeto:

```
Invoke-RestMethod -Method Post -Uri http://127.0.0.1:8000/api/evento -ContentType 'application/json' -Body '{"aluno_id":"corpo-1","tipo_evento":"queda","localizacao":"sala-12"}'
```

Deve responder com o `id` e os hashes, e o e-mail “[Auditix] Possível queda” chega em segundos. Na primeira vez ele costuma cair no **spam**: marque “não é spam” uma vez e os próximos vão direto para a caixa de entrada.

Não use `curl.exe` com JSON no PowerShell.

Ele mexe nas aspas antes de o comando sair, o corpo chega vazio e o servidor responde `422`.

Depois, o teste de verdade: abra a Sala e **deite no chão passando da linha**. Tem que dar alerta na tela, **um** e-mail, e o evento no painel com o hash.

9 Quando não funciona

A janela do servidor diz o que houve. Procure por estas linhas:

LINHA	O QUE FAZER
<code>webhook falhou</code>	A <code>WEBHOOK_URL</code> está errada ou sem internet.
<code>webhook recusou (403)</code>	O segredo do <code>.env</code> e o da Lambda estão diferentes.
<code>webhook recusou (502)</code>	A função quebrou. O erro real está em Lambda → Monitorar → Exibir logs do CloudWatch , no fluxo mais recente.
nenhuma linha, e nada chega	O servidor entregou e a AWS aceitou: olhe o spam do Gmail e, depois, o CloudWatch.

No CloudWatch, os erros possíveis e o que cada um quer dizer:

ERRO	CAUSA
<code>HandlerNotFound</code>	O código não foi colado, ou faltou clicar em Deploy .
<code>KeyError: 'REMETENTE'</code>	Variável de ambiente vazia na Lambda.
<code>AccessDenied</code>	Falta anexar <code>AmazonSESFullAccess</code> ao papel da função.
<code>Email address is not verified</code>	O e-mail foi verificado numa região diferente da função (ela está em <code>us-east-1</code>).

10 Duas coisas de segurança

Um servidor por vez. Se duas máquinas rodarem apontando para a mesma Lambda, cada queda vira dois e-mails: a espera de 60 segundos é contada dentro de cada servidor, e um não sabe do outro.

Segredo e senha se trocam. O segredo da Lambda apareceu em prints durante a montagem, e a senha do banco também. Trocar os dois leva cinco minutos: gere um valor novo com `uv run python -c "import secrets;print(secrets.token_urlsafe(32))"`, atualize a variável `SEGREDO` na Lambda e o `.env`, e reinicie o servidor. Enquanto o segredo antigo valer, quem o tiver dispara e-mails em nome da escola — e a fatura é de vocês.

Resumo do que tem que aparecer quando está tudo certo

PostgreSQL conectado · alerta por e-mail LIGADO (graves, espera 60s) · a Sala carregando as 6 etapas · a linha do chão marcada · um e-mail por queda.

Auditix AI — grupo 11 · Desafio DATEN x FIAP, “Antes do NEXT”. Guia de instalação; os detalhes da montagem na AWS estão em `aws/LEIA.md` e os da câmera em `CAMERA-IP.md`, dentro do projeto.